COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/07/2026	**DATE SUBMITTED:**	10/15/2026
NAME:	JAMES DANIEL M. VERANO	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 1 **Basic C++ Programming**

1. PROCEDURE OUTPUT

The procedures that I performed to solve every specific ILO are all focused on the **basic fundamentals of C++ programming language**. With the previous knowledge of my **Programming Logic and Design (PLD),** It helped me do the activity with ease. With the knowledge I have **preserved** and **maintained** in my previous year, it helped me further recall and initialize functions with troubles. With that, let's get to them one by one:

ILOA - In this activity ILO, It consists of the basic essentials of **variable declaration** and **arithmetic** and **logical operations**. It brings back the creation of functions where you have to use **function prototypes, function definition**, and **operations** to return a value that satisfies the intended C++ code program in a cleaner and strict manner. Such as in the first function, you take 2 numbers, define its parameters in the function prototype, and create the function's method in the function definition, and call the function to get the sum of that program. As for the rest, all it takes is a boolean approach where you compare the logical operations whether the results will be true or false.

ILOB - In this ILO, it consists of a class declaration, to separate the **private and public section** of the **code declaration** in the **function prototype**. Where the private section is where variables and declarations initialized there will only be accessed within the class, while public sections are declared codes that can be accessed even outside of the class function. With this, it allows you to prevent messing up the code structure and you can easily identify the errors within the code. In the ILOB it talks about validating the overall parameters given in terms of if the triangle is valid or not. Which only implies that you only need to see if it is <= 180.

ILOC - The last and most heavy activity and time consuming of all, the supplementary activity, where it consists of the **modification of ILOB, swapping of variables, conversion of temperature**, and **calculation of distance between 2 points**. With the previous ILO's they already have provided a guidance structure of how **class declaration, function prototypes**, and **function definition** deeply helps in creating the structure of the program. Thus, it was easy as you only need the formulas to declare their intended **variables, parameters, methods, process**, and **initialization**. So, with the guidance of the structure, I flawlessly gathered the formulas I needed and created first the needed parameters and variables to initialize them. Afterwards, I **created the methods, formula calculation,** the **if-else statements for the modified ILOB**, and the **overall structure** needed in the main function to fully collect the needed parameters for the **condition to be met**.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

[ILO A: Create Code that follows the Basic C++ Code Structure]

[SOURCE CODE]

```cpp
#include <iostream>

// FUNCTION DECLARATIONS
int sumOfTwoNum(int fNum, int sNum); // Number 1
bool isGreater(int a, int b); // Number 2
bool displayLogicalOperations(bool c, bool d);

// MAIN PROGRAM

int main () {

    // ILO A #1
    std::cout << "The sum of all numbers is : " << sumOfTwoNum(10,22) << "\n" << std::endl;

    // ILO A #2
    int x = 500;
    int y = 400;
   if (isGreater(x, y)) {
        std::cout << "x is greater than y" << "\n" << std::endl;
    } else {
        std::cout << "x is not greater than y" << "\n" << std::endl;
    }

    // ILO A #3
    bool success = displayLogicalOperations(true, false);
    std::cout << "Function execution status: " << std::boolalpha << success << std::endl;

    return 0;
}
```

```cpp
// FUNCTIONS
// ILO A #1
int sumOfTwoNum(int fNum, int sNum){
    int sum = fNum + sNum;
    return sum;
}
 // ILO A #2
bool isGreater(int a, int b){
    return a > b;
}
// ILO A #3
bool displayLogicalOperations(bool c, bool d) {
    // std::boolalpha formats outputs as "true" or "false" instead of "1" or "0"
    std::cout << std::boolalpha;

    std::cout << " || Logical Operations || " << std::endl;
    std::cout << "--------------------------" << std::endl;
    std::cout << "Inputs: A = " << c << ", B = " << d << std::endl;
    std::cout << "--------------------------" << std::endl;

    // 1. Logical NOT
    std::cout << "NOT A (!A)      : " << (!c) << std::endl;
    std::cout << "NOT B (!B)      : " << (!d) << std::endl;

    // 2. Logical AND
    std::cout << "LOGICAL AND (&&): " << (c && d) << std::endl;

    // 3. Logical OR
    std::cout << "LOGICAL OR (||) : " << (c || d) << std::endl;

    // 4. Logical XOR
    std::cout << "LOGICAL XOR (^) : " << (c ^ d) << std::endl;

    return true;
}
```

[OUTPUT]

```
The sum of all numbers is : 32

x is greater than y

 || Logical Operations ||
--------------------------
Inputs: A = true, B = false
--------------------------
NOT A (!A)      : false
NOT B (!B)      : true
LOGICAL AND (&&): false
LOGICAL OR (||) : true
LOGICAL XOR (^) : 1
Function execution status: true
PS C:\Users\Striker\Documents\CPE21S4\CPE010>
```

[ILOB: Implement Appropriate Class Definition and Instances Based on Given Requirements]

[SOURCE CODE]

HOA 1.1 > ILOB.cpp > setAngles(double, double, double)

```cpp
#include <iostream>

class Triangle {
    private:
        double totalAngle, angleA, angleB, angleC;

    public:
        Triangle(double A, double B, double C);
        const bool validateTriangle();
        void setAngles(double A, double B, double C);
};

int main() {

Triangle setl(40, 20, 110);
    if(setl.validateTriangle()) {
        std::cout << "------------------------------" << std::endl;
        std::cout << "the shape is a valid triangle.\n";
        std::cout << "------------------------------" << std::endl;
    } else {
        std::cout << "----------------------------------" << std::endl;
        std::cout << "The shape is NOT a valid triangle. \n";
        std::cout << "----------------------------------" << std::endl;
    }

    return 0;
}

Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}
```

```
35
36  void Triangle::setAngles(double A, double B, double C){
37      angleA = A;
38      angleB = B;
39      angleC = C;
40      totalAngle = A+B+C;
41  }
42
43  const bool Triangle::validateTriangle(){
44      return (totalAngle <= 180);
45  }
```

[OUTPUT]

```
------------------------------
the shape is a valid triangle.
------------------------------
PS C:\Users\Striker\Documents\CPE21S4\CPE010> ^C
PS C:\Users\Striker\Documents\CPE21S4\CPE010>
PS C:\Users\Striker\Documents\CPE21S4\CPE010>  &
din=Microsoft-MIEngine-In-enkuegcz.v2g' '--stdout
nsy.xve' '--dbgExe=C:\msys64\mingw64\bin\gdb.exe'
---------------------------------
The shape is NOT a valid triangle.
---------------------------------
```

Sections	Answer (ILO-A)
Header File Declaration Section	`#include <iostream>`
Global Declaration	None

Class Declaration and Method Declaration Section	`int sumOfTwoNum(int fNum, int sNum); // Number 1`
	`bool isGreater(int a, int b); // Number 2`
	`bool displayLogicalOperations(bool c, bool d);`

Main Function	`int main () {`
	
	`  // ILO A #1`
	`  std::cout << "The sum of all numbers is : " << sumOfTwoNum(10,22) << "\n" << std::endl;`
	`  // ILO A #2`
	`  int x = 500;`
	`  int y = 400;`
	`  if (isGreater(x, y)) {`
	`  std::cout << "x is greater than y" << "\n" << std::endl;`
	`  } else {`
	`  std::cout << "x is not greater than y" << "\n" << std::endl;`
	`  }`
	`  // ILO A #3`
	`  bool success = displayLogicalOperations(true, false);`
	`  std::cout << "Function execution status: " << std::boolalpha << success << std::endl;`
	
	`  return 0;`
	`}`

Method Definition	`// ILO A #1`
	`int sumOfTwoNum(int fNum, int sNum){`
	`  int sum = fNum + sNum;`

```
    return sum;
}
 // ILO A #2
bool isGreater(int a, int b){
    return a > b;
}
// ILO A #3
bool displayLogicalOperations(bool c, bool d) {
    // std::boolalpha formats outputs as "true" or "false"
instead of "1" or "0"
    std::cout << std::boolalpha;

    std::cout << " || Logical Operations || " << std::endl;
    std::cout << "-------------------------" << std::endl;
    std::cout << "Inputs: A = " << c << ", B = " << d <<
std::endl;
    std::cout << "-------------------------" << std::endl;

    // 1. Logical NOT
    std::cout << "NOT A (!A)      : " << (!c) << std::endl;
    std::cout << "NOT B (!B)      : " << (!d) << std::endl;

    // 2. Logical AND
    std::cout << "LOGICAL AND (&&): " << (c && d) <<
std::endl;

    // 3. Logical OR
    std::cout << "LOGICAL OR (||) : " << (c || d) <<
std::endl;

    // 4. Logical XOR
    std::cout << "LOGICAL XOR (^) : " << (c ^ d) <<
std::endl;

    return true;
```

	}

Sections	Answer (ILO-B)
Header File Declaration Section	`#include <iostream>`
Global Declaration	`none`

Class Declaration and Method Definition Section	`class Triangle {`
	`  private:`
	`  double totalAngle, angleA, angleB, angleC;`
	
	`  public:`
	`  Triangle(double A, double B, double C);`
	`  const bool validateTriangle();`
	`  void setAngles(double A, double B, double C);`
	`};`

Main Function	`int main() {`
	
	`Triangle setl(40, 20, 110);`
	`  if(setl.validateTriangle()) {`
	`  std::cout << "----------" << std::endl;`
	`  std::cout << "the shape is a valid triangle.\n";`
	`  std::cout << "----------" << std::endl;`
	`  } else {`
	`  std::cout << "----------" << std::endl;`
	`  std::cout << "The shape is NOT a valid triangle. \n";`
	`  std::cout << "----------" << std::endl;`
	`  }`
	
	`  return 0;`
	`}`

Method Definition	`Triangle::Triangle(double A, double B, double C){`

```
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}

const bool Triangle::validateTriangle(){
    return (totalAngle <= 180);
}
```

OBSERVATION FOR ILO B:

- The code structure is very well constructed and implemented. Starting from the **function prototype**, or **class declaration** in other terms, we can see that we have **declared variables** in the **private section**. The private section defines that the variables will only be accessed inside the class, and can only be manipulated inside the class. While the public can be controlled even outside the class structure. In the **public section**, contains the caller's variable and the parameters for the variables to be taken inside the **main function**. Below in the main function, are the methods. They are the **function definition**. They define the process and the computation that happens when the parameter is met within the condition in the **class declaration**. The class declaration as observed, consist of the a procedure;

    - Where it validates if the parameter given is a **Triangle** = *Triangle::validateTriangle()*
    - A computation for the **totalAngle** = *Triangle::Triangle()*
    - **Void setAngle** to change the variable values = *Triangle::setAngles()*

  Lastly, is the **main function**. It consists of the very structure of the UI, an if-else statement function initialized to a specific condition, IF the certain parameter given in the variable **Triangle setI()** is a valid triangle.

[SUPPLEMENTARY ILOC: Solve Different Problems using the C++ Programming Language]

[SOURCE CODE OF ILO C, ACTIVITY NUMBER 1, 2, 3]

```cpp
#include <iostream>
#include <cmath>
#include <string>

double kelvinToFahrenheit(double kelvin);

double calculateDistance(double x1, double y1, double x2, double y2);

int main() {

    double num1, num2, temp, kelvin, x1, y1, x2, y2, a, b, c;

    std::cout << "Enter the first number: ";
    std::cin >> num1;
    std::cout << "Enter the second number: ";
    std::cin >> num2;

    std::cout << "\n--- Before Swapping ---" << std::endl;
    std::cout << "First number = " << num1 << std::endl;
    std::cout << "Second number = " << num2 << std::endl;

    // 3. Swapping using a temporary variable
    temp = num1;   // Store the value of num1 in temp
    num1 = num2;   // Copy the value of num2 into num1
    num2 = temp;   // Copy the value of temp (original num1) into num2

    // Display of values after swapping
    std::cout << "\n--- After Swapping ---" << std::endl;
    std::cout << "First number = " << num1 << std::endl;
    std::cout << "Second number = " << num2 << std::endl;



    std::cout << "\nEnter temperature in Kelvin: ";
    std::cin >> kelvin;
```

```cpp
    double fahrenheit = kelvinToFahrenheit(kelvin);

    std::cout << kelvin << "k is equal to " << fahrenheit << " Fahrenheit." << std::endl;



    std::cout << "\nEnter coordinates of the first point (x1 y1): ";
    std::cin >> x1 >> y1;
    std::cout << "Enter coordinates of the second point (x2 y2): ";
    std::cin >> x2 >> y2;

    double distance = calculateDistance(x1, y1, x2, y2);
    std::cout << "The distance between the two points is: " << distance << std::endl;

    return 0;
}

double kelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9.0 / 5.0 + 32;
}


double calculateDistance(double x1, double y1, double x2, double y2) {
    return std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
}
```

THIS SOURCE CODE COVERS:

1. **Create a C++ program to swap the two numbers in different variables.**

Function Prototype	**None**
Main Function	**Line Code 13 - 20:** The user input for the value of the variables to be swapped. **Line Code 23 - 25:** The process of swapping the 2 variables and the initialized declarations of variables. **Line Code 28 - 30:** The outputting process of the swapping of variables.
Function Definition	**None**

2. **Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.**

Function Prototype	**Line Code 5:** Declaration of parameter for the conversion and process of kelvin variable.
Main Function	**Line Code 34 - 39:** The User Input interface, the initialization of the variable's value, the calling of the method process, and the outputting of the fahrenheit value of the kelvin.
Function Definition	**Line Code 54 - 55:** The method process of the kelvin to fahrenheit conversion, where it consists of the calculation formula of converting the kelvin temperature to fahrenheit.

3. **Create a C++ program that has a function that will calculate the distance between two points.**

Function Prototype	**Line Code 7:** The declaration of the x and y value coordinates, and the specified parameters assigned to them.
Main Function	**Line Code 43 - 49:** The declaration and initialization of the x and y values, and the calling of the function definition of the calculation of distance between 2 points. In the last line of codes, is the outputting of the fahrenheit value.
Function Definition	**Line Code 59 - 60:** Consist of the function definition of the calculation formula for solving the distance between 2 points.

[SOURCE CODE OF ACTIVITY 4 MODIFIED ILO B]

```cpp
#include <iostream>
#include <cmath>
#include <string>

double computeArea(double a, double b, double c);
double computePerimeter(double a, double b, double c);
std::string determineTriangle(double a, double b, double c);

int main () {

    double a, b, c;

    std::cout << "------------------------------" << std::endl;
    std::cout << "Enter side A: ";
    std::cin >> a;
    std::cout << "Enter side B: ";
    std::cin >> b;
    std::cout << "Enter side C: ";
    std::cin >> c;
    std::cout << "------------------------------" << std::endl;

    double area = computeArea(a, b, c);
    double perimeter = computePerimeter(a, b, c);
    std::string triangleType = determineTriangle(a, b, c);

    std::cout << "\nTriangle Information:" << std::endl;
    std::cout << "Area: " << area << std::endl;
    std::cout << "Perimeter: " << perimeter << std::endl;
    std::cout << "Type: " << triangleType << "\n" << std::endl;

    return 0;
}
```

HOA 1.1 > modified ILOB.cpp > main()

```cpp
double computeArea(double a, double b, double c) {
    double s = (a + b + c) / 2; // semi-perimeter
    return std::sqrt(s * (s - a) * (s - b) * (s - c)); // Area's formula
}
double computePerimeter(double a, double b, double c) {
    return a + b + c;
}
std::string determineTriangle(double a, double b, double c) {
    double longestSide, side1, side2;

    if (a >= b && a >= c) {
        longestSide = a;
        side1 = b;
        side2 = c;
    } else if (b >= a && b >= c) {
        longestSide = b;
        side1 = a;
        side2 = c;
    } else if (c >= a && c >= b) {
        longestSide = c;
        side1 = a;
        side2 = b;
    } else {
        return "Invalid triangle sides";
    }

    double longestSquare = pow(longestSide, 2);
    double sumOfSquares = pow(side1, 2) + pow(side2, 2);

    if (longestSquare == sumOfSquares) {
        return "Right-angled triangle";
    } else if (longestSquare < sumOfSquares) {
        return "Acute-angled triangle";
    } else {
        return "Obtuse-angled triangle";
    }
}
```

ACTIVITY 4 CODE ANALYSIS:

- As in the structure of the code, we are tasked to create and initialize a function method and declaration that computes the area, perimeter, and to determine the type of triangle it is in according to the given value in the parameter. As stated in the **function prototype**, we have 3 declared variables, **computeArea, computePerimeter,** and **std::string determineTriangle**; they are given a specific parameter, which all correlates to the same value as they are talking about the same triangle. In the function definition, are the methods of the declared variables, to be specific, the functionality of the function prototypes. It contains the process and computation of all the essentials needed stated in the ILO for number 4. As shown in the code above, **Line Code 34 - 70**, are the formulas and conditions inside certain functions;

  computeArea() = Consist and initializes the formula for calculating the area of the triangle using the parameter's value given by the user.
  computePerimeter() = Consist and initializes the formula in calculating the perimeter of the triangle.
  determineTriangle() = Consist of a structured if-else statement that contains a specific condition according to the given triangle side values, where in it identifies which is the longest side (highest value), and that value will be initialized to another variable named **longestSide**. After the initialization, it will be initialized to another variable, but processed into **pow()** to find the sum of squares and the longest square. After that, another if-else statement structure will be in process after the initialization of the new variable. It's the if-else statement where it identifies the identity it fits in according to the calculated **longestSquare** and **sumOfSquares**.

  In the main program, as always it is the user interface where the side values are taken from the user, where the functions are called and processed, and lastly, where the output is displayed. As shown above, l**ine 22 - 23,** indicates the calling of function with their given parameter. And that is sent into the function definition.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google's AI overview.**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Formulas and code assistance for ILO C #4 letter C.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I explored options between how to determine the way of modification of ILO B code structure to have a better insight of the code implementation and function structure.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.